

NYC Multi-Camera Vehicle Tracking

Technical Research Brief and Infrastructure Plan

Target scope	1,000 nearby Manhattan traffic cameras
Source cadence	One public snapshot approximately every 15 seconds
Core objective	Detect vehicles, perform short-lived cross-camera re-identification, and visualize probable routes
Retention rule	Delete source frames immediately; expire vehicle-level sessions after no more than one hour

Prepared as a consolidated export of the project research and design discussion

1. Executive Summary

This project is a recruiter-facing computer-vision and distributed-systems demonstration. It ingests public traffic-camera snapshots, detects and crops vehicles, generates temporary appearance embeddings, and attempts to continue a vehicle session across nearby cameras using road connectivity, travel-time windows, direction, class, color, and visual similarity. The project should emphasize confidence-aware inference rather than claiming perfect identification.

- Use nearby Manhattan cameras arranged as directed road-connected clusters, not a citywide all-to-all search.
- Process source images in memory and delete them immediately after inference.
- Keep at most one or two representative crops, compact embeddings, route events, and confidence metadata per active vehicle session.
- Expire vehicle-level data after one hour at the absolute latest, and sooner when tracking confidence is lost.
- Do not collect or retain license-plate text, plate hashes, faces, owner information, or stable long-term identifiers.
- Demonstrate system quality through latency, throughput, ambiguity handling, false-match controls, and live route visualization.

2. Core Assumptions and Capacity Calculations

Parameter	Assumption
Cameras	1,000 within Manhattan
Refresh interval	15 seconds per camera
Average source image size	199 KB
Retention	Source frame deleted after inference; vehicle sessions $\leq$ 1 hour
Traffic topology	Cameras grouped into road-connected clusters

Metric	Calculated load
Images per second	$1,000 / 15 = 66.7$
Images per minute	4,000
Images per hour	240,000
Images per day	5.76 million
Incoming bandwidth	~ 13.3 MB/s
Raw bytes processed per hour	~ 47.8 GB
Raw bytes processed per day	~ 1.15 TB

The raw daily volume is processed, not stored. The architecture must avoid turning this transient stream into an object-storage workload.

3. Vehicle-Detection Workload

Average visible vehicles per image	Vehicle detections per second	Detections per hour
3	~ 200	720,000
6	~ 400	1.44 million
10	~ 667	2.40 million

15	~1,000	3.60 million
----	--------	--------------

A realistic planning range is approximately 6-10 usable detections per image, but this must be benchmarked on the selected cameras. Re-identification should not run on every repeated detection; it should run when a track is new, the crop improves materially, a vehicle reaches an exit zone, or a handoff is expected.

4. Inputs

4.1 Camera registry

Field	Purpose
camera_id	Stable internal camera identifier
image_url	Current public JPEG snapshot URL
latitude / longitude	Map placement and road-graph construction
street / intersection	Human-readable display
camera direction	Northbound, southbound, eastbound, westbound, or calibrated bearing
refresh interval	Expected cadence, nominally 15 seconds
image dimensions	Coordinate mapping and quality checks
source timestamp	Camera-provided timestamp when available, otherwise fetch timestamp
authorization metadata	Terms, permitted uses, attribution, and display restrictions

```
{
  "cameraId": "nyc_cam_1842",
  "imageUrl": "https://source.example/current.jpg",
  "latitude": 40.7580,
  "longitude": -73.9855,
  "direction": "northbound",
  "street": "7th Avenue",
  "intersection": "West 47th Street",
  "refreshSeconds": 15
}
```

4.2 Directed camera-transition graph

Camera proximity alone is insufficient. The system needs a directed road graph describing which camera transitions are physically plausible.

Field	Example
from_camera	cam_1842
to_camera	cam_1851
minimum travel time	18 seconds
maximum travel time	85 seconds
direction	northbound
route distance	240 meters
transition probability	0.72

- OpenStreetMap road geometry
- One-way streets and turn restrictions
- Camera orientation and field of view
- Historical or estimated travel times
- Manual validation for the demonstration corridor

5. End-to-End Processing Pipeline

Camera scheduler

- > HTTP fetch
- > ETag / Last-Modified / byte-hash duplicate filter
- > JPEG decode and resize in memory
- > batched vehicle detector
- > crop vehicles in memory
- > camera-local association
- > selective Re-ID embedding
- > road-graph candidate filtering
- > cluster-local similarity matching
- > one-hour TTL session store
- > API / WebSocket updates
- > interactive map dashboard
- > immediate source-frame deletion

5.1 Ingestion and duplicate suppression

1. Spread camera requests uniformly across each 15-second cycle instead of polling all 1,000 cameras at once.
2. Use HTTP ETag and Last-Modified headers where supported.
3. If metadata is unavailable, hash response bytes to identify exact duplicate frames.
4. Use perceptual hashing only when camera overlays or timestamps change while the scene remains effectively unchanged.
5. Place accepted frames into a short batching queue and retain them only in memory.

5.2 Preprocessing

- Decode JPEG once.
- Resize the longest side to 640-960 pixels, selected by benchmark.
- Preserve coordinate transforms back to the source image.
- Use pinned memory and asynchronous GPU transfer where practical.
- Reject stale, corrupt, or unexpectedly small frames before GPU work.

5.3 Vehicle detection

- YOLO-family detector exported to ONNX or TensorRT
- FP16 initially; calibrated INT8 after validating accuracy
- Batch size approximately 16-32 for the 66.7-images-per-second target
- 100-250 ms maximum batching delay, acceptable against a 15-second source cadence

```
{
  "cameraId": "cam_1842",
  "timestamp": 1783844002,
  "boundingBox": [420, 180, 730, 460],
  "vehicleClass": "sedan",
  "detectorConfidence": 0.95
}
```

5.4 Camera-local association

Because each camera provides a snapshot only every 15 seconds, normal high-frame-rate tracking assumptions do not apply. Local association should combine position, lane region, direction, appearance, class, and temporal continuity. The output is best described as snapshot-to-snapshot association rather than conventional continuous tracking.

5.5 Crop selection and Re-ID embeddings

- Crop every detected vehicle in memory.
- Compute a crop-quality score using area, sharpness, occlusion, angle, and detector confidence.
- Keep only the best representative crop and optionally the latest crop for UI rendering.
- Generate an embedding when a vehicle first appears, when quality improves materially, or before a likely handoff.
- Discard redundant crops immediately.

5.6 Cross-camera matching

Vehicle exits CAM-A

- > query reachable cameras
- > open expected arrival windows
- > receive new detections from CAM-B/C/D
- > filter by direction, class, color, and time
- > compare temporary embeddings
- > accept only when confidence and margin are high
- > otherwise end or mark the session uncertain

Signal	Illustrative weight
Appearance embedding similarity	45-50%
Travel-time plausibility	20%
Road-route compatibility	12-15%
Direction/lane consistency	5-10%
Vehicle class	5-7%
Approximate color	5-8%

```
if best_score >= 0.90 and (best_score - second_best_score) >= 0.12:
    continue_session()
else:
    end_or_mark_uncertain()
```

A false negative is preferable to falsely merging two unrelated vehicles. The UI should expose uncertainty and alternative candidates rather than hiding it.

6. Outputs

6.1 User-facing selected-vehicle view

- Temporary session ID
- Vehicle class and approximate color
- Current camera and last confirmed sighting
- Session duration and expiration countdown
- Cameras traversed in the current temporary session
- Per-handoff confidence
- Probable route with uncertain segments visually distinguished
- Rejected or alternative candidates for transparency

```
{
  "temporaryId": "T-204",
  "vehicleClass": "sedan",
  "color": "dark-gray",
  "currentCamera": "cam_1851",
  "sessionDurationSeconds": 142,
  "camerasObserved": 3,
  "currentMatchConfidence": 0.934,
  "routeConfidence": "high",
  "status": "active"
}
```

6.2 Operational dashboard

Metric	Why it matters
Active cameras	Source health
Images processed per second	Pipeline throughput
Duplicate frames skipped	Cost reduction
Vehicle detections per second	Model workload
GPU utilization	Capacity efficiency
Median/p95 detection latency	Performance
Median/p95 match latency	User responsiveness
Active temporary sessions	State pressure
Cross-camera handoffs per minute	Core capability
Ambiguous-match rate	Model honesty
Track-loss rate	Quality
Cost per million frames	Infra efficiency

6.3 Long-term aggregate analytics

Long-term storage should contain only aggregates without persistent vehicle identifiers.

```
{
  "cameraId": "cam_1842",
  "windowStart": "2026-07-12T12:00:00Z",
  "windowMinutes": 5,
  "vehicleCount": 81,
  "sedanCount": 46,
  "truckCount": 12,
  "busCount": 4,
  "northboundCount": 67,
  "averageDetectionConfidence": 0.92
}
```

7. Infrastructure

7.1 Recommended language split

Layer	Preferred language	Rationale
Camera scheduler and HTTP ingestion	Go	Excellent concurrency, operational simplicity, fast development
Control-plane APIs and WebSocket/SSE	Go	Strong networking and straightforward deployment
JPEG decode, resize, crop hot path	Rust	Tighter memory control, zero-copy opportunities, SIMD-friendly processing
Cluster-local matcher/index	Rust	Predictable latency and efficient in-memory data structures
Model experimentation and evaluation	Python	Best ecosystem for training and research
Production inference	TensorRT or ONNX Runtime	GPU-optimized deployment independent of Python request handling

The GPU is likely to dominate inference time. The language split should be justified by profiling, not aesthetics. A Go-only MVP is valid; Rust should be introduced where measurements show meaningful gains.

7.2 Services and technologies

Component	Suggested technology
Camera scheduler / fetchers	Go
Internal queue	NATS JetStream or Redis Streams
Image preprocessing	Rust
Vehicle detection	TensorRT
Re-ID inference	TensorRT or ONNX Runtime
Camera metadata	PostgreSQL
Road graph	PostgreSQL + PostGIS
Active sessions	Redis / Valkey
Vector matching	HNSW or FAISS per geographic cluster
Temporary crops	Local NVMe or short-lived cache
Public API	Go
Live updates	WebSocket or Server-Sent Events
Frontend	Next.js / React
Map	MapLibre GL

Metrics	Prometheus + Grafana
Edge/CDN	Cloudflare

7.3 Geographic clustering

The 1,000 cameras should be divided into approximately 10-20 road-connected clusters, usually 50-150 cameras each. Matching remains local to the cluster and to the outgoing transition edges of a camera.

Manhattan

- | - Lower Manhattan
- | - Midtown South
- | - Times Square / Midtown West
- | - Midtown East
- | - Upper West Side
- | - Upper East Side
- ` - Harlem

7.4 Suggested deployment layout

Node	Responsibilities
Control plane	Go scheduler, fetchers, API, health checks, camera registry access
GPU worker	Rust preprocessing, TensorRT detection, selective Re-ID embeddings
State/matching node	Redis/Valkey, HNSW/FAISS shards, transition-window matcher
Cloudflare edge	Frontend delivery, CDN, authentication, rate limits, public API protection

For an MVP, the control-plane and state/matching services can share one server. A second GPU-capable machine can handle all inference. Production-style redundancy can be added later.

8. GPU and Throughput Planning

The detector target is approximately 66.7 images per second before duplicate suppression. A single optimized inference GPU may be enough, depending on model size, resolution, batch size, and scene density. A two-GPU deployment offers redundancy and room for Re-ID.

Deployment	Use
1 GPU	Development and non-redundant MVP
2 GPUs	Recruiter-facing production demo with redundancy or split detection/Re-ID
3 GPUs	Overflow, experimental model, or standby capacity

- Benchmark at 640 and 960 input sizes.
- Measure end-to-end throughput rather than relying on vendor FPS claims.
- Track GPU utilization, queue depth, p95 latency, and cost per million frames.
- Use FP16 first; move to INT8 only after calibration confirms acceptable accuracy.

9. Storage and Data Lifecycle

Data	Retention
Source snapshot	Milliseconds; delete immediately after inference
Failed/corrupt snapshot	Delete immediately
Rejected or duplicate crop	Delete immediately
Best representative crop	Until session ends, maximum one hour
Latest crop for UI	Optional; maximum one hour
Appearance embedding	Until session ends, maximum one hour
Temporary route history	Until session ends, maximum one hour
Temporary session ID	Until session ends, maximum one hour
Aggregate traffic statistics	Long term
License-plate text or plate hash	Do not collect
Faces	Blur or discard where visible

```
TTL = min(  
    one_hour_from_first_sighting,  
    session_loss_timeout  
)
```

A 512-dimensional FP16 embedding is approximately 1 KB before metadata and index overhead. A practical active-track record without a crop may be 2-4 KB. A representative WebP crop may add roughly 15-50 KB. The system should never retain every repeated detection crop.

10. Privacy, Safety, and Product Boundaries

Hashing a license plate does not make a vehicle route anonymous; it creates a stable pseudonymous identifier. The privacy-preserving design is therefore based on temporary appearance-based sessions, strict TTLs, no plate collection, no owner lookup, and no persistent historical search.

- No license-plate OCR or stable plate-derived IDs.
- No facial recognition or owner identification.
- No cross-day or permanent vehicle histories.
- No public search by plate, make/model, home area, or repeated location pattern.
- No manual endpoint to extend a session beyond its TTL.
- No download/export of individual route histories.
- Use clear language such as probable route, candidate match, and last confirmed sighting.
- Keep audit logs for administrative access and configuration changes.

The safest persistent cross-camera demonstration uses synthetic or consenting vehicles. For real public feeds, keep tracking ephemeral and uncertainty-aware.

11. Major Technical Risks

Risk	Impact	Mitigation
15-second snapshot gaps	Cars may skip cameras or appear only once	Use road-graph windows and present probabilistic results
Dense Manhattan traffic	Many visually similar vehicles	Use topology, timing, direction, confidence margins, and conservative rejection
Occlusion by buses/pedestrians	Lost or low-quality crops	Quality scoring and track

		termination rather than forced matching
Lighting/camera variation	Embedding drift	Domain-specific evaluation, normalization, and camera-aware calibration
Stale/duplicate feeds	Wasted GPU work	ETag, Last-Modified, byte hash, and source-health monitoring
Incorrect camera orientation	Bad route candidates	Manual validation and calibration tooling
Source terms or feed changes	Legal/operational interruption	Maintain source registry, permissions metadata, and fallback datasets
False merges	Misleading vehicle history	High thresholds, best-vs-second-best margin, and visible uncertainty

12. Optimization Priorities

Optimization	Expected benefit
Skip unchanged frames	Reduces network decode and GPU work
Resize before inference	Major GPU reduction
Batch 16-32 images	Higher GPU utilization
FP16 / calibrated INT8	Lower inference cost
Selective Re-ID	Potentially 80-95% fewer embedding calls
Keep one best crop	Large storage reduction
Road-graph candidate filtering	Massive reduction in vector comparisons and false matches
Cluster-local indexes	Lower memory and latency
Store only aggregates long term	Minimal persistent storage
Scale from 50 -> 250 -> 1,000 cameras	Avoids premature infrastructure spend

13. Rollout Plan

Stage	Camera count	Approx. image rate	Primary goals
Stage 1	50	3.3 images/sec	Source reliability, ingestion, detection, map UI, basic local association
Stage 2	250	16.7 images/sec	Road graph, candidate windows, Re-ID evaluation, TTL cleanup
Stage 3	1,000	66.7 images/sec	Geographic sharding, redundancy, monitoring, cost reporting, polished recruiter demo

- Collect and validate the camera registry and usage permissions.
- Build one carefully chosen corridor with manually verified transition edges.
- Benchmark detector accuracy and throughput at 640 and 960 pixels.
- Create a labeled evaluation set for cross-camera matches and non-matches.
- Implement temporary session lifecycle and enforced deletion before public release.

11. Add road-graph matching and ambiguity-aware confidence scoring.
12. Build the dashboard around both visual outputs and engineering metrics.
13. Scale only after measuring cost per million frames and false-match behavior.

14. Recruiter-Facing Positioning

Suggested project name: CrossView - Real-Time Multi-Camera Vehicle Re-Identification.

A distributed computer-vision system that detects, tracks, and re-identifies vehicles across nearby urban camera views using appearance embeddings, temporal constraints, and graph-based route inference. Real-world feeds use temporary identifiers and strict retention limits, while the dashboard exposes model uncertainty, system throughput, latency, and cost.

- Highlight multi-camera Re-ID, road-graph reasoning, distributed ingestion, GPU batching, and uncertainty handling.
- Show accepted and rejected candidates with confidence scores.
- Publish benchmark methodology and failure cases.
- Demonstrate privacy-aware engineering as a product-quality decision, not a disclaimer.
- Emphasize measured throughput and cost efficiency rather than merely maximizing camera count.

15. Open Decisions and Validation Work

- Exact public camera source, access terms, and whether automated computer-vision processing is permitted.
- Actual Manhattan camera count available under the selected source and geographic density of useful views.
- Average image resolution, actual refresh reliability, and duplicate-frame rate.
- Average usable vehicles per frame and typical crop dimensions.
- Best detector and Re-ID model after evaluation on the selected camera domain.
- Whether one GPU sustains the target with acceptable p95 latency.
- How long a session can remain unmatched before termination; the one-hour value is a hard maximum, not necessarily the normal duration.
- Camera graph generation method and amount of manual validation required.
- Public UI access controls and whether selected crops can legally be redisplayed.

16. Source and Reference Links

The following resources were discussed during research. Access terms, pricing, and documentation can change; verify them before implementation.

- **NYC DOT data feeds:** <https://www.nyc.gov/html/dot/html/about/datafeeds.shtml>
- **511NY developer resources:** <https://511ny.org/developers/resources>
- **EarthCam Times Square public cameras:** <https://www.earthcam.com/usa/newyork/timesquare/>
- **EarthCam API:** <https://www.earthcam.net/api/>

- **Windy Webcams API:** <https://api.windy.com/webcams/docs>
- **TrafficLand API:** <https://www.trafficland.com/api.html>
- **NIST Privacy Engineering - Disassociability:** <https://www.nist.gov/itl/applied-cybersecurity/privacy-engineering/collaboration-space/focus-areas/disassociability/tools>
- **FTC: hashing does not make data anonymous:** <https://www.ftc.gov/policy/advocacy-research/tech-at-ftc/2024/07/no-hashing-still-doesnt-make-your-data-anonymous>
- **Cloudflare R2 pricing:** <https://developers.cloudflare.com/r2/pricing/>
- **AWS EC2 G6 instances:** <https://aws.amazon.com/ec2/instance-types/g6/>
- **Google Cloud Run pricing:** <https://cloud.google.com/run/pricing>
- **Qdrant pricing:** <https://qdrant.tech/pricing/>